The Islamic University of Gaza
Faculty of Engineering
Department of Computer Engineering

ECOM 2013 - Combinational Digital Design

Lab #2

Boolean Algebra

Eng. Rania R. Alsayed   Eng. Hashem M. Hejazy

Spring 2025/2026

Introduction

Mathematical methods that simplify circuits rely primarily on **Boolean algebra.** It deals with variables that can take on two possible values: true (1) or false (0).

Basic Theorems

Some Simplification Theorem

- **$x + 0 = x$**
- **$x.0 = 0$**
- **$x + x = x$**
- **$x . x = x$**
- **$x + 1 = 1$**
- **$x.1 = x$**
- **$x + x' = 1$**
- **$x.x'= 0$**

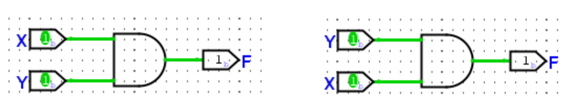
De Morgan’s Laws:

De Morgan's Laws can be applied to simplify complex logical expressions by pushing the negation inward and changing the logical operator. By using these laws, we can often reduce the number of terms and make the expression more manageable or easier to understand.

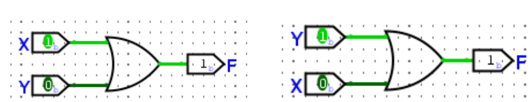
- $(X + Y)' = X'Y'$
- $(XY)' = X' + Y'$

Commutative Law

- $XY = YX$



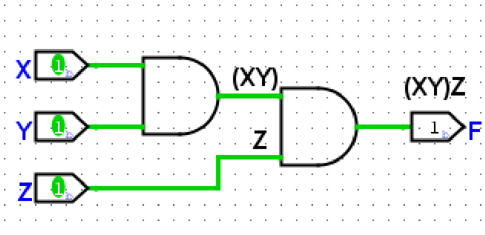
- $X + Y = Y + X$

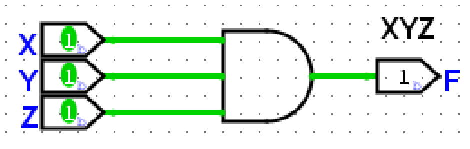


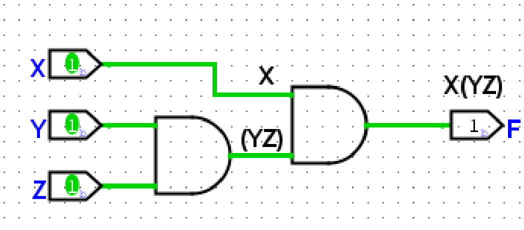
Associative Law:

The Associative Law allows us to rearrange the grouping of logical operations within a Boolean expression without changing its overall meaning. This property is particularly useful when simplifying complex Boolean expressions or when designing logical circuits.

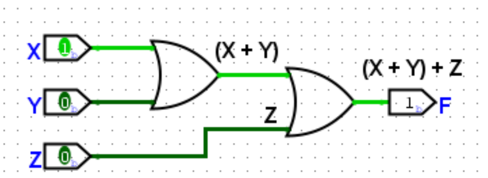
❖ $(XY)Z = X(YZ) = XYZ$

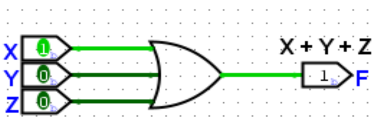


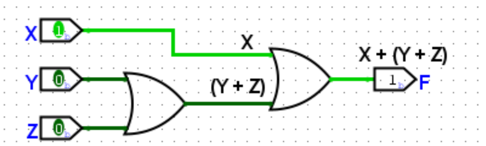




❖ $(X + Y) + Z = X + (Y + Z) = X + Y + Z$







Distributive Law

The Distributive Law allows us to expand or collapse Boolean expressions by distributing the logical operations across terms. It is particularly useful in simplifying complex Boolean expressions, reducing redundancy, and optimizing logical circuits.

- ❖ $X(Y + Z) = XY + XZ$
- ❖ $X + YZ = (X + Y)(X + Z)$

Absorption Law

The Absorption Law allows for the simplification of Boolean expressions by eliminating redundant terms. By applying this law, we can reduce the complexity of logical expressions and make them easier to analyze and manipulate.

- ❖ $X + XY = X$
  - ❖ **Steps:**

    $X+XY$ (Factor out X)
    $=X(1+Y)$ (Use the Boolean identity: $(1+Y)=1$)
    $=X\cdot 1$ (Apply identity law: $(X\cdot 1)=X$)
    $=X$ (**Final Result:** $X+XY=X$)

- ❖ $X(X + Y) = X$
  - ❖ **Steps:**

    $X(X + Y)$ (Apply the **Distributive Law**)
    $= XX + XY$ (Since $X \cdot X = X$)
    $=X+ XY$ (Apply the **Absorption Law** ($X + XY = X$))
    $=X$ (**Final Result:** $X(X + Y) = X$)

Consensus Law

- **$XY + X'Z + YZ = XY + X'Z$**
  - Steps:
    1. **Start with the left-hand side (LHS):**
       $XY+X'Z+YZ$

    2. **Factor using distributive property:**
       Look at the terms **YZ** and see if it can be combined with the other terms:

       $XY+X'Z+YZ=XY+X'Z+YZ(X+X')$

    Explanation: $X+X'=1$, so multiplying by it does not change the expression.

    3. **Distribute YZ over (X + X'):**

$XY+X'Z+YZX+YZX'$

    4. **Reorder terms (for clarity):**

$XY+YZX+X'Z+YZX'$

    5. **Apply the absorption law:**

- $XY+YZX=XY$
- $X'Z+YZX'=X'Z$

    6. **Final simplified expression (Right-hand side, RHS):**

$XY+X'Z$

This proves the **Consensus Law**: $XY+X'Z+YZ=XY+X'Z$

- **$(X + Y)(X' + Z)(Y + Z) = (X + Y)(X' + Z)$**

Elimination Law

- **$X + X'Y = X+Y$**
  - Steps:

    $X + X'Y$ (Factor using the distributive property)

    $=(X+X').(X+Y)$ (Since $(x+x')=1$

    $= 1.(X+Y)$

    $= X+Y$

- **$X(X' + Y) = XY$**
  - Steps:

    $X(X' + Y)$ (Distribute X over the sum (Distributive Law))

    $= X \cdot X'+XY$ (since $X \cdot X'=0$)

    $=0+XY$

    $=XY$

Uniting Law

- **$XY + XY' = X$**
  - Steps

    $XY + XY'$ (Factor out X)

    $= X(Y+Y')$ (since $Y+Y'=1$)

    $=X.1$

    $=X$

- **$(X + Y)(X + Y') = X$**
  - Steps:

    $(X + Y)(X + Y')$ (Apply the distributive law)

    $=XX+XY'+YX+YY'$ (Since $X.X=X$ and $YY'=0$ and $XY'+YX=X$(uniting low))

    $=X+X+0$

    $=X$

❖Cost Criteria

- **Cost criteria** is used to measure the simplicity of a logic circuit.

- **Gate cost** is simply the number of gates used to implement an expression.

- **Literal cost** is the number of literals in an expression.

- **Input cost** is the number of literals plus the number of terms in an expression

❖ Example of equations in sum-of-products form:

$F1 = A C' + A B' D + A' B' C + A' C D' + B' C' D'.$

❖ Examples of equations in product-of-sums form are:

$F2 = (A'+ C'+ D) (A + B'+ C) (A + C + D') (B'+ C'+ D').$

- ❖ **Ex: Derive the Boolean expression from the following truth table then simplify its minimum number of literals manually and Find the cost criteria then use Logisim to check your answer**

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Solution:

The Boolean expression is taken from the combinations where F = 1, so

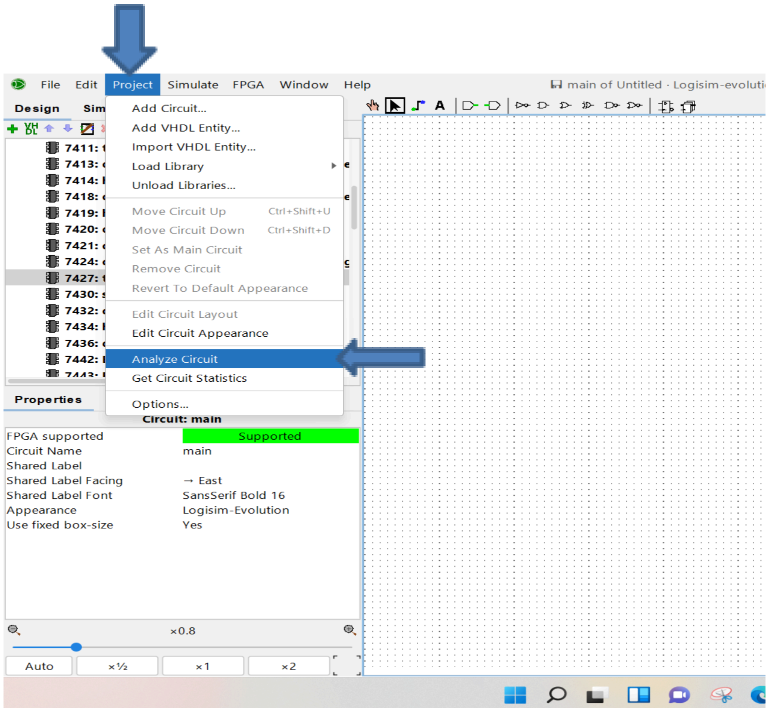
F = A'BC + AB'C' + AB'C + ABC' + ABC(Using Uniting law)
F = A'BC + AB' + AB (Again, using Uniting law)
F = A'BC + A (using Elimination law)
F = BC + A

Cost criteria:

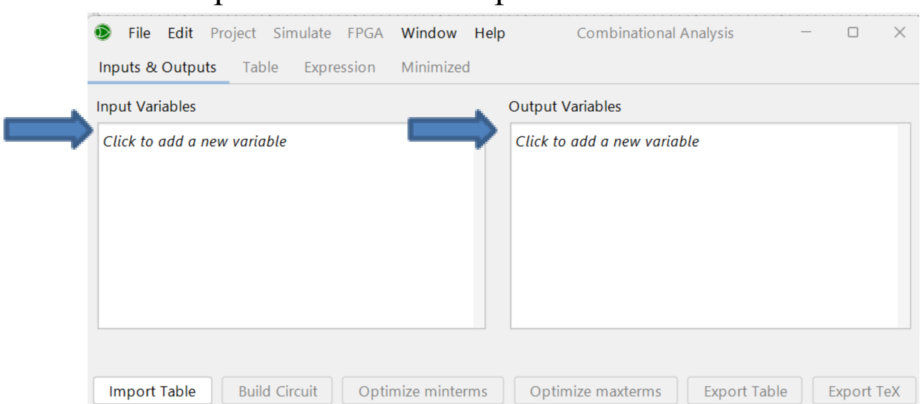
Cost criteria	Before Simplification	After Simplification
Gate cost	6	2
Literal cost	15	3
Input cost	20	4

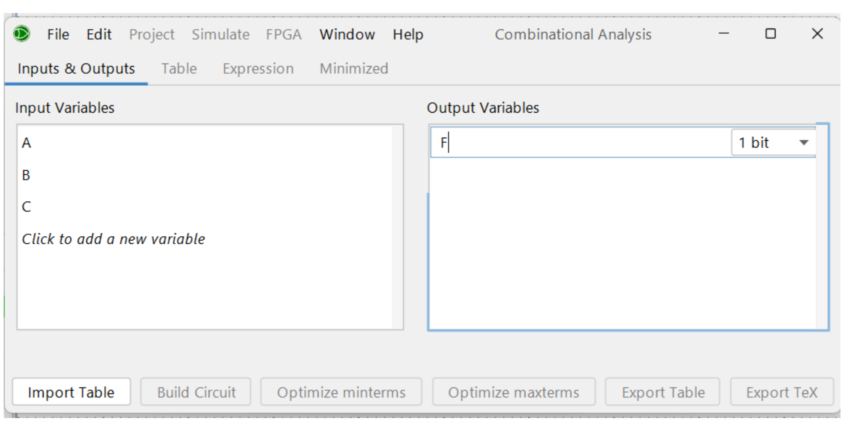
Simplify Truth table using Logisim :

1. From menu bar choose Project then Analyze Circuits

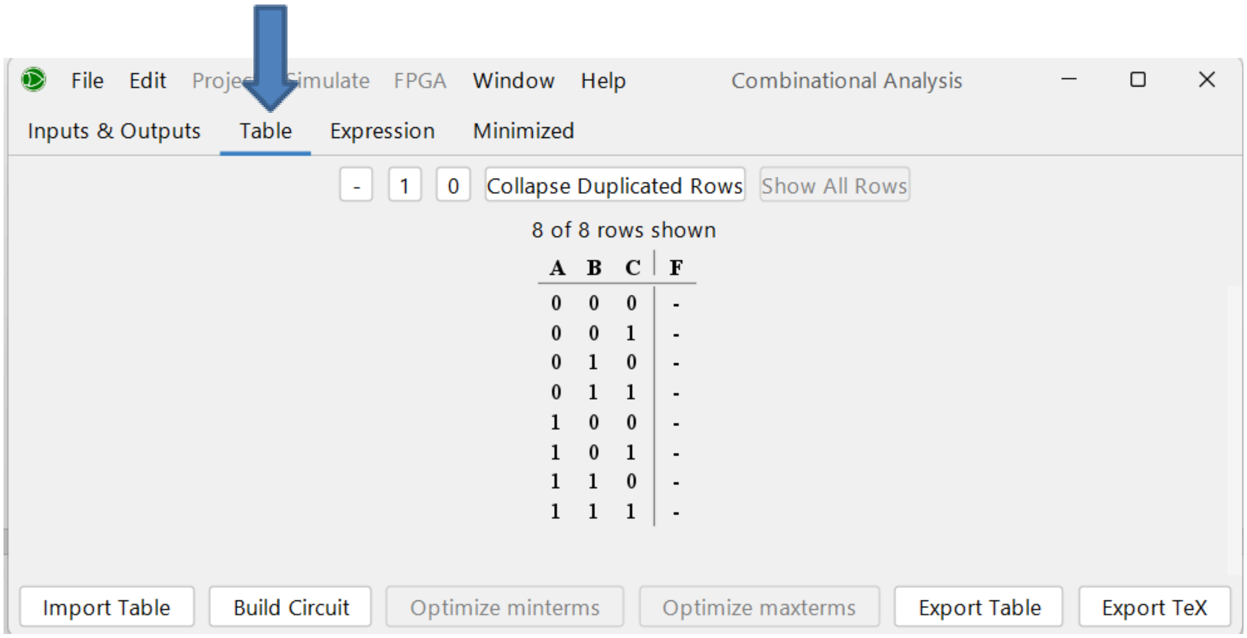


2. Add inputs Name and output

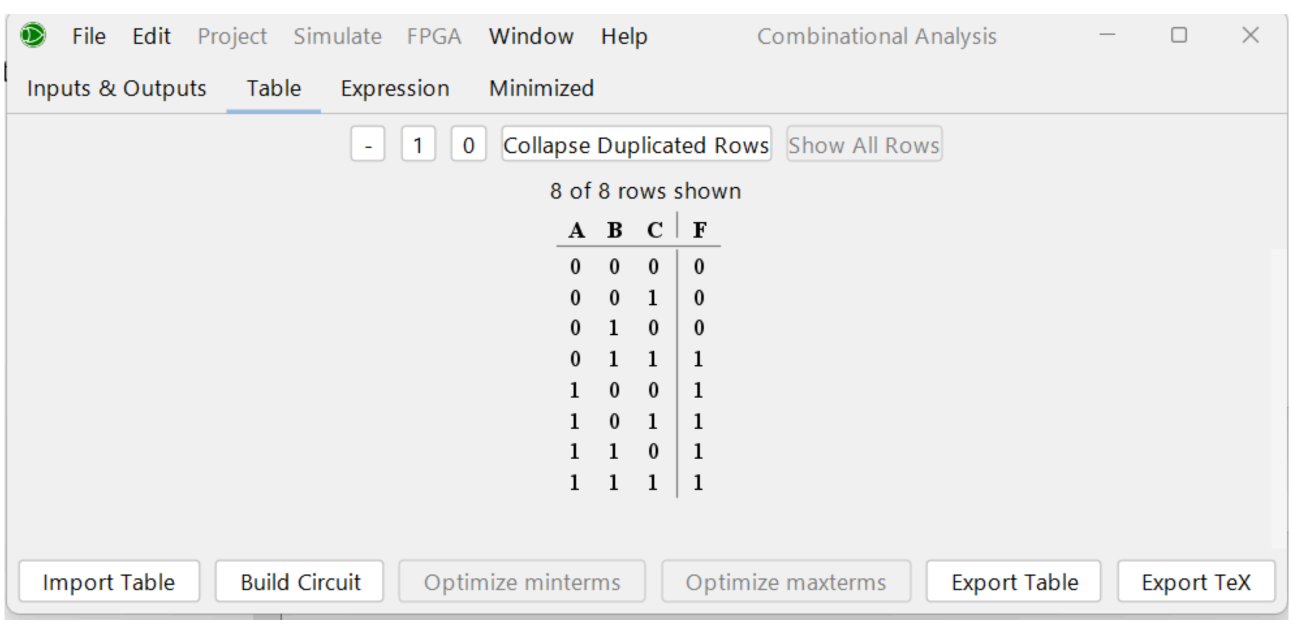




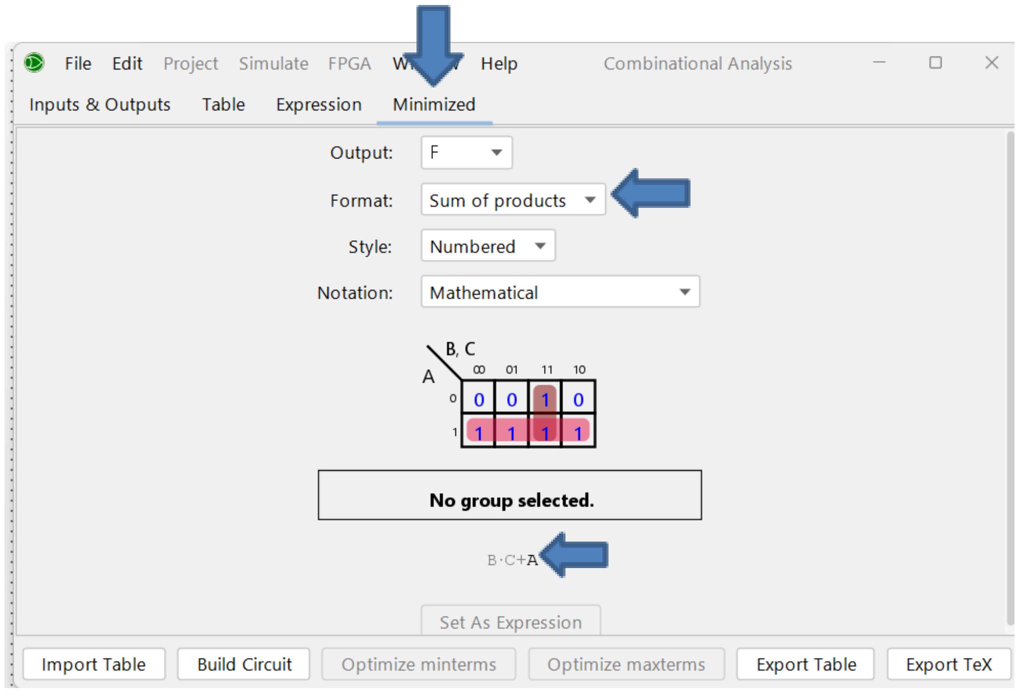
3. Click on table To add the Truth Table



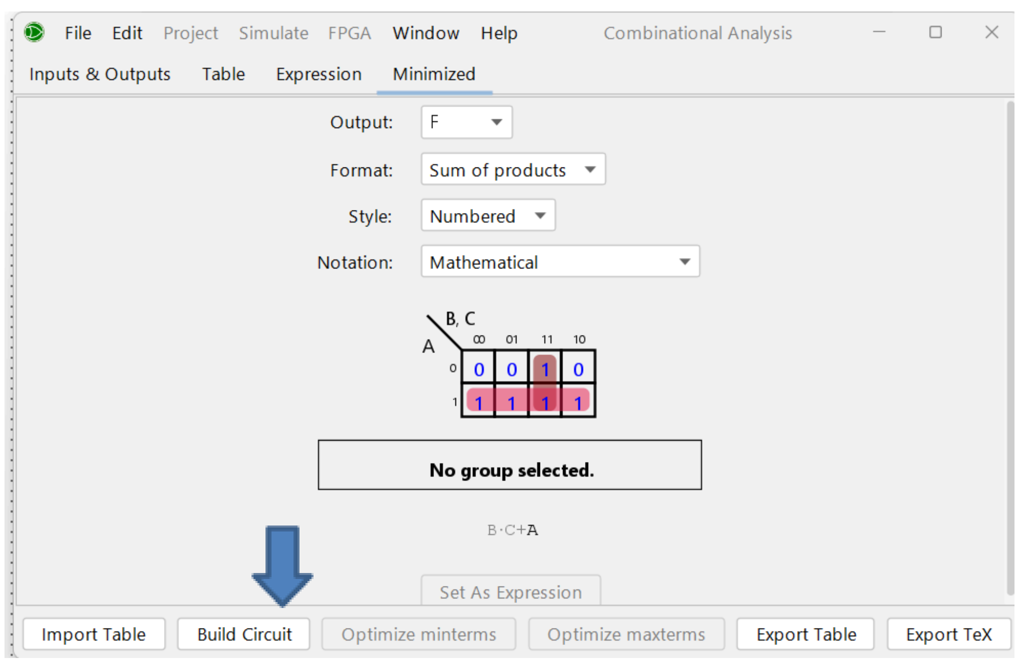
Then complete the Truth Table :



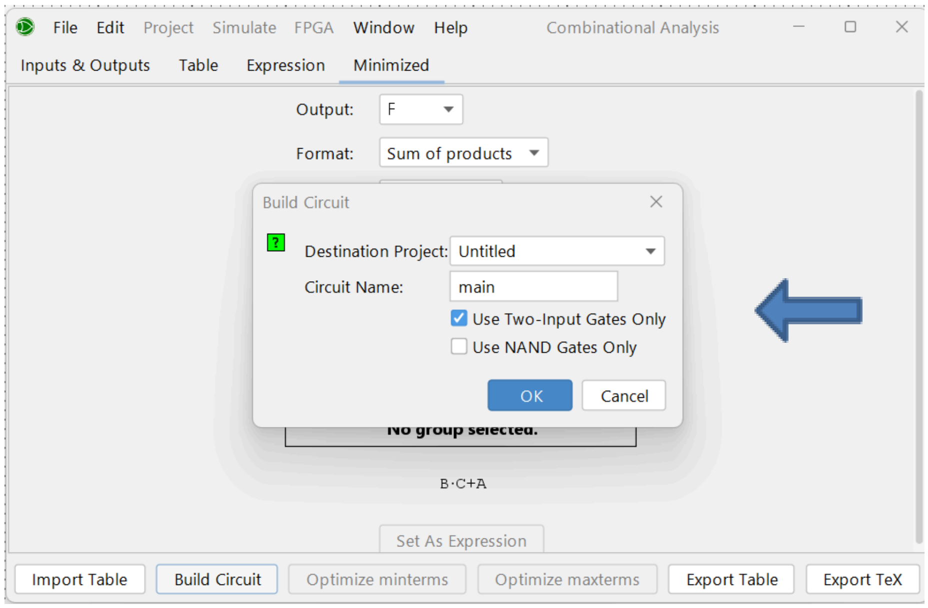
4. From Minimized we can choose if we want formal sum of product or product of sum, we can see the result



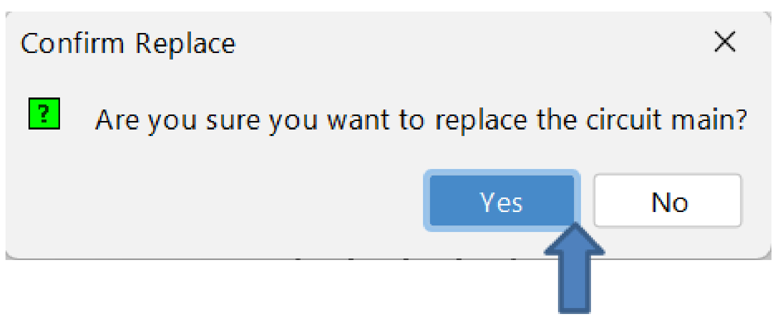
5. Click on Build Circuit to draw it on logisim



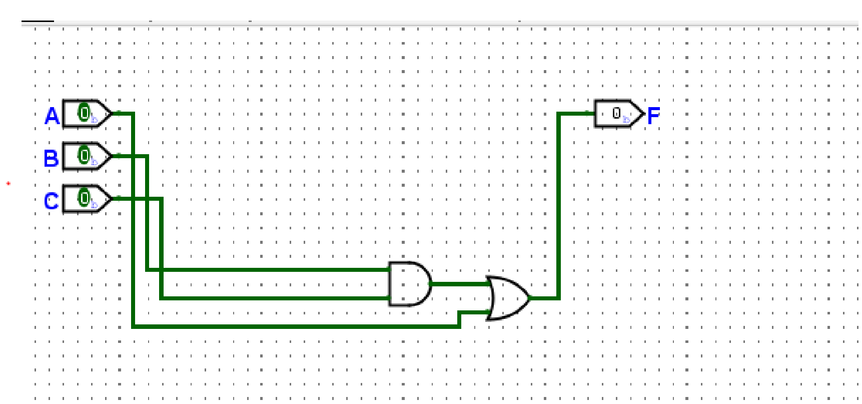
6. Choose Use Two-Input Gates Only



7. Click on Yes:



8. The Circuit will appear in Logisim :



<u>Ex:</u> Simplify the following Boolean expressions to a minimum number of literals manually and Find the cost criteria then use Logisim to check your answer

F = ABC'D + A'BD + ABCD

<u>Solution:</u>

F = ABC'D + A'BD + ABCD (Using Uniting Law ABC'D+ABCD=ABD)

= ABD+A'BD (Using Uniting Law ABD+A'BD =BD)

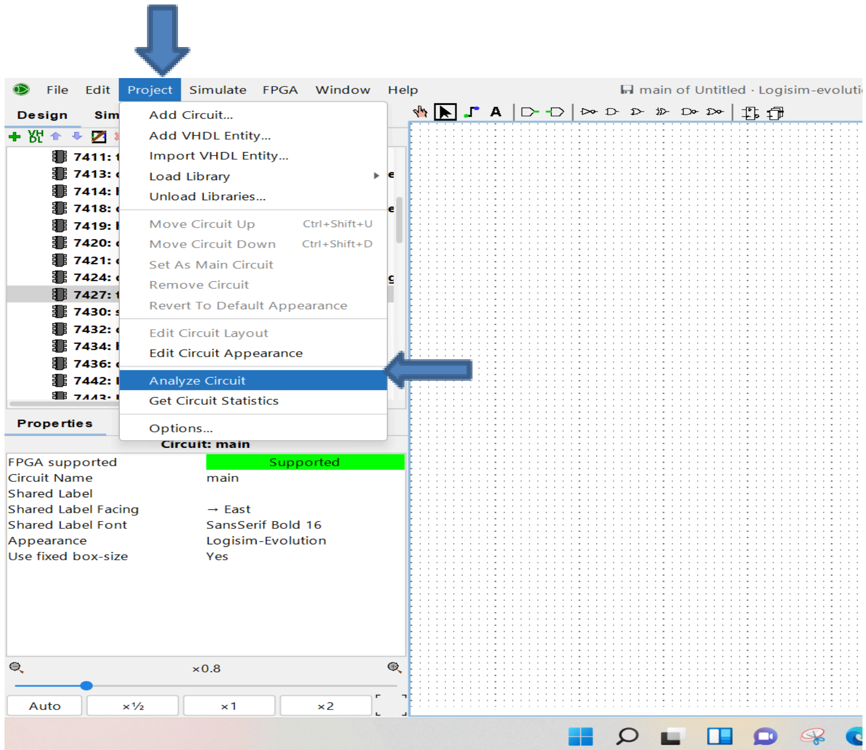
= BD

Cost criteria:

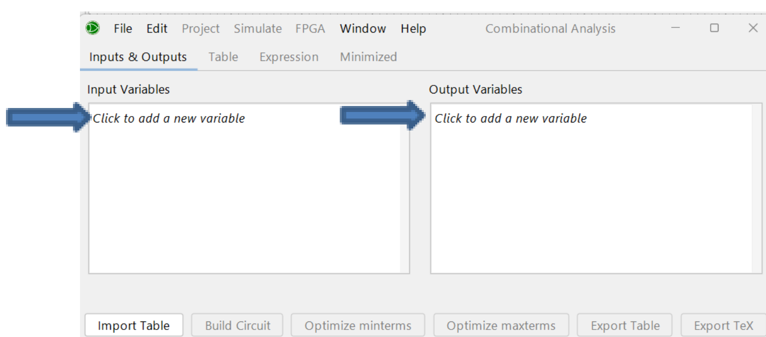
Cost criteria	Before Simplification	After Simplification
Gate cost	4	1
Literal cost	11	2
Input cost	14	2

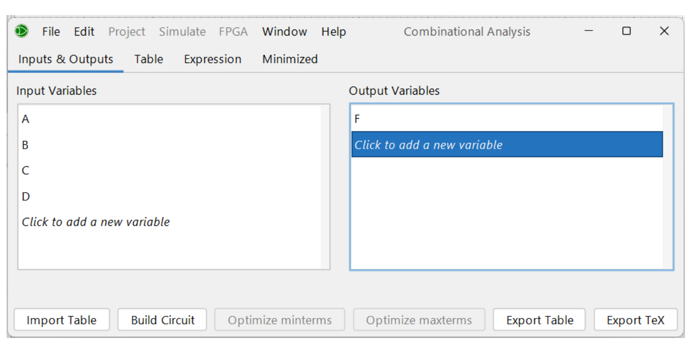
Simplify Expression using Logisim :

1. From menu bar choose Project then Analyze Circuits

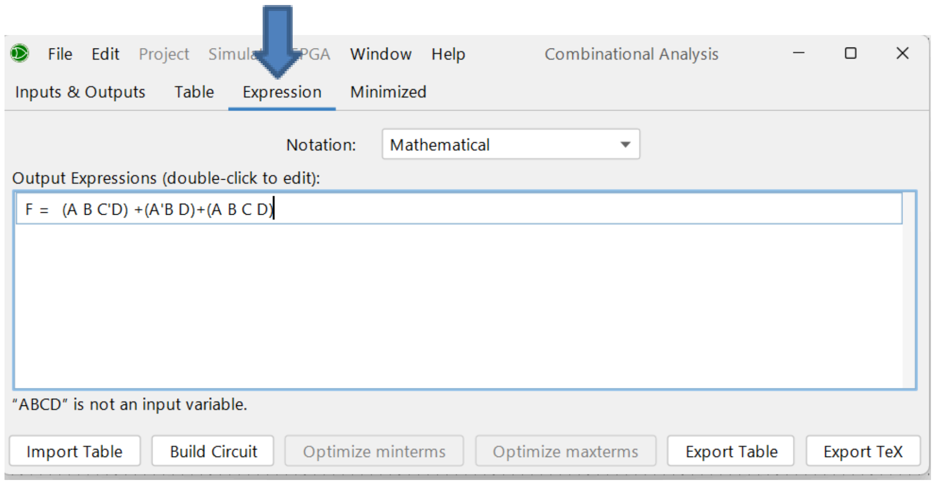


2. Add inputs Name and output

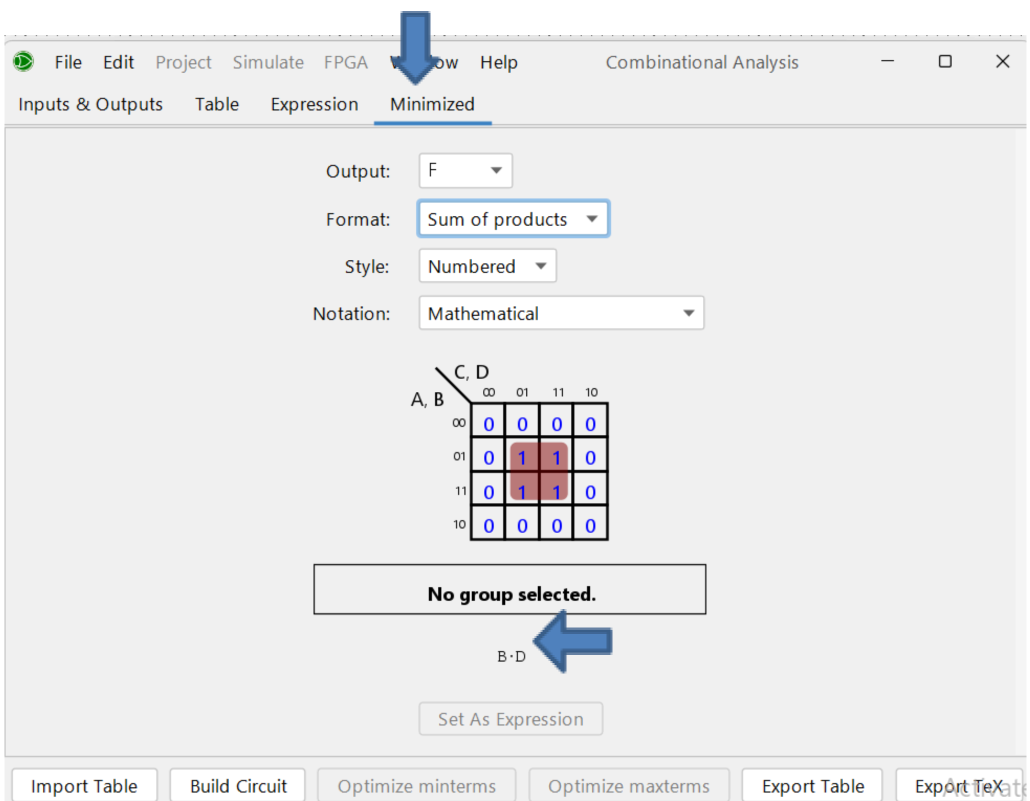




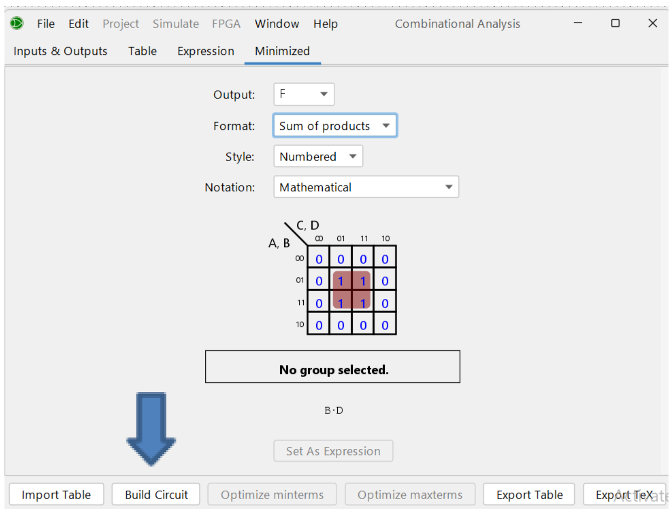
3. **Click on Expression to enter Expression then click Enter (make space between letter and put parentheses)**



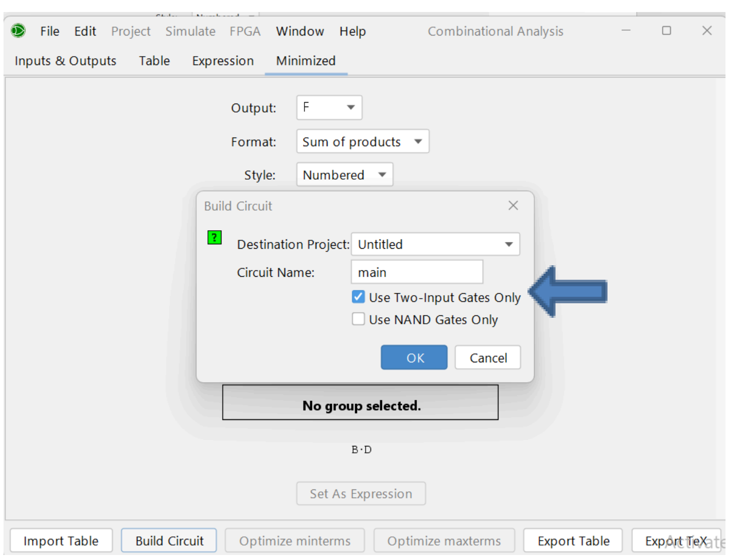
4. **Click on Minimized the result appear (we can choose if we want formal sum of product or product of sum)**



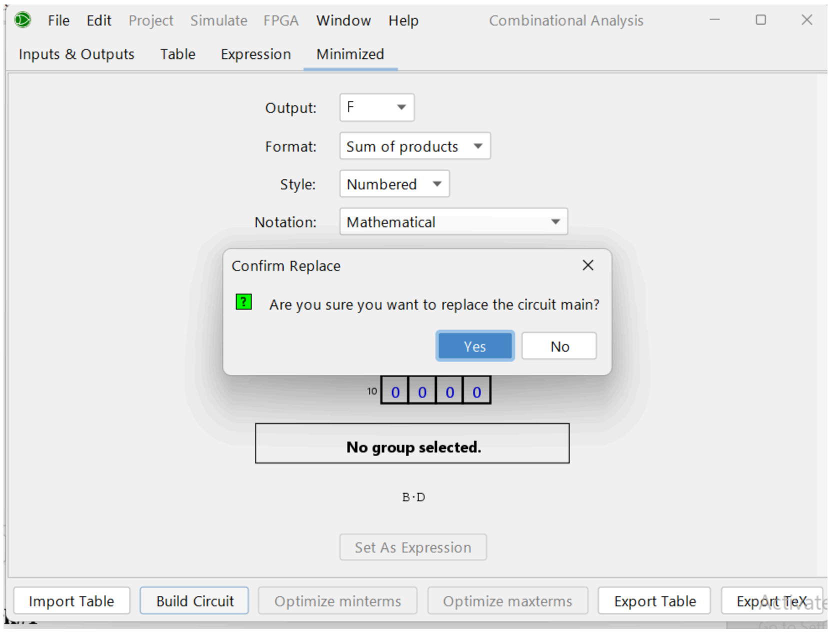
5. To draw it on logisim click on Build Circuit



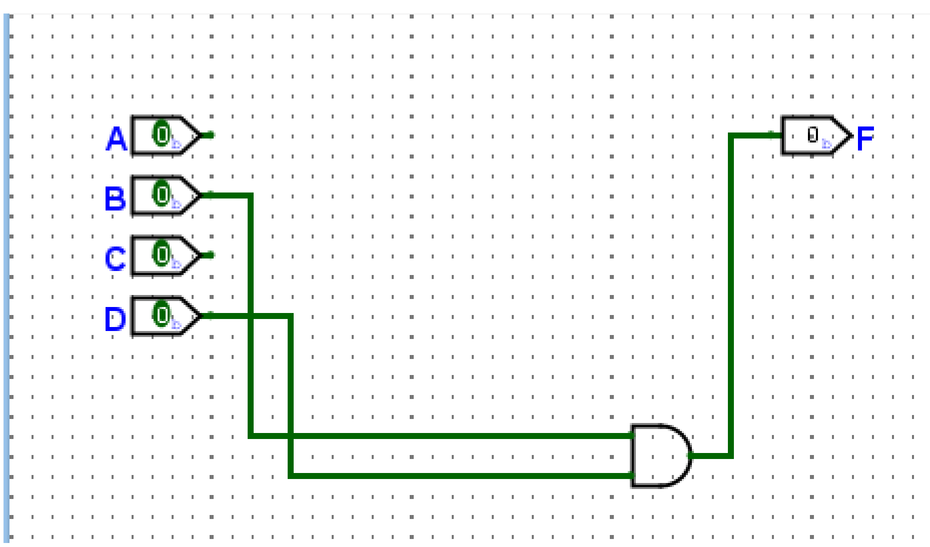
6. Choose Use Two-Input Gates Only:



7. Click on Yes :



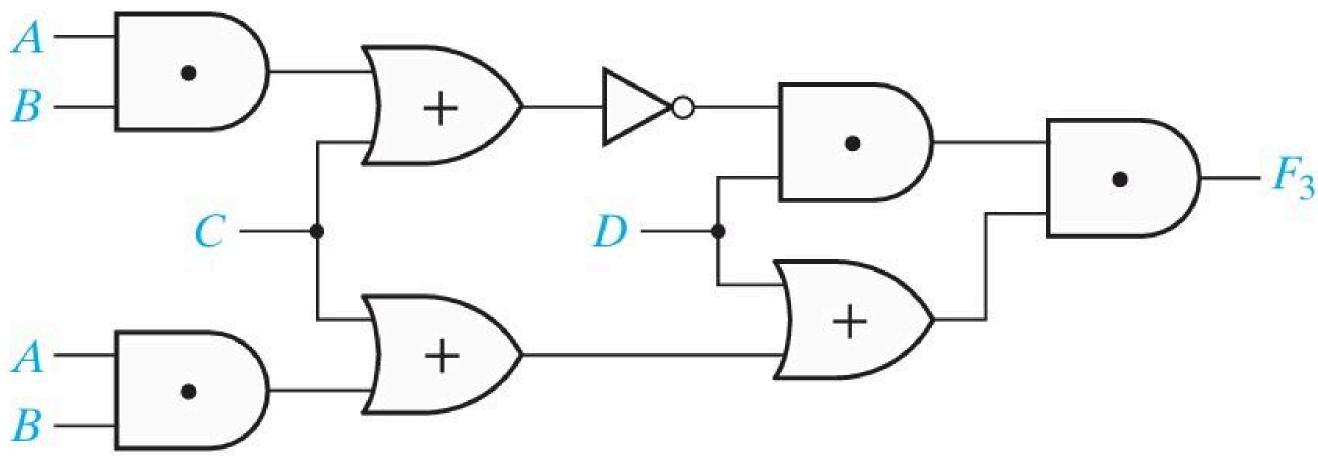
8. The Circuit will appear in Logisim :



Homework#

Ex#1:

For the circuit below, find the output (Hint: Find the circuit output by first finding the output of each gate, going from left to right, and simplifying as you go) simplify it manually then Check your answer using logisim and build Circuit(or boolean Algebra Calculator for mobile)



Ex#2:

Derive the Boolean expression from the following truth table, then simplify it to the minimum Sum of Products (SOP) form manually.
After that, find the cost criteria and finally use Logisim (or boolean Algebra Calculator for mobile)to verify your answer and to build the circuit

Truth Table

A	**B**	**C**	**F**
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Ex#3:

Simplify the following expressions to a minimum sum of products :
$((A + B')C)'(C + A)' +(A + B)$
Manually then use Logisim (or boolean Algebra Calculator for mobile)to verify your answer and to build the circuit